

DevSecOps: Técnicas e Ferramentas de Segurança na Automação de Pipelines CI/CD

Arthur E. B. Xavier¹, Déborah T. Santos¹, Gustavo A. Silva¹

¹Universidade Federal de Rondonópolis (UFR)
Rondonópolis – MT – Brazil

{arthur.xavier, deborah.teles, gustavo.silva}@aluno.ufr.edu.br

Abstract. *The evolution of agile development demands the continuous integration of security via DevSecOps and Shift Left. This article analyzes how test automation (SAST, DAST, and SCA) mitigates vulnerabilities in CI/CD pipelines through a literature review based on the Kitchenham method and the PICOS model, which selected seven studies. The results indicate that automation reduces deployment time to minutes, while Artificial Intelligence (AI) and Natural Language Processing (NLP) reduce incident response time by 45% and production failures by 60%. Furthermore, although LLMs detect 90% to 100% of vulnerabilities, they generate excessive false positives and alert fatigue. It is concluded that the success of DevSecOps does not depend on the accumulation of market tools, but on the calibration of specific rules and hybrid models that align technical precision with team usability.*

Resumo. *A evolução do desenvolvimento ágil exige integrar a segurança continuamente via DevSecOps e Shift Left. Este artigo analisa como a automação de testes (SAST, DAST e SCA) mitiga vulnerabilidades em pipelines CI/CD por meio de uma revisão de literatura baseada no método de Kitchenham e modelo PICOS, que selecionou sete estudos. Os resultados indicam que a automação reduz o tempo de deploy para minutos, enquanto a Inteligência Artificial (IA) e o Processamento de Linguagem Natural (NLP) reduzem em 45% o tempo de resposta a incidentes e em 60% as falhas em produção. Além disso, embora os LLMs detectem de 90% a 100% das vulnerabilidades, eles geram excesso de falsos positivos e fadiga de alertas. Conclui-se que o êxito do DevSecOps não depende do acúmulo de ferramentas de mercado, mas da calibração de regras específicas e de modelos híbridos que alinhem precisão técnica à usabilidade das equipes.*

Palavras-chaves: Integração, Pipeline CI/CD, Shift Left, DevSecOps, SAST, Vulnerabilidades, Desenvolvimento Seguro

1. Introdução

A rápida evolução dos ciclos de desenvolvimento de software impõe a necessidade de integrar a segurança de forma contínua, superando os modelos tradicionais que tratam a proteção como uma etapa final isolada. A adoção da metodologia DevSecOps transforma essa dinâmica ao incorporar práticas automatizadas desde as fases iniciais do ciclo de vida, mitigando vulnerabilidades antes que o código atinja o ambiente de produção. Ao promover essa mudança de paradigma, conhecida como "Shift Left", as organizações

conseguem antecipar a detecção de falhas e garantir a integridade dos ativos críticos sem comprometer a velocidade da entrega contínua.

O objetivo fundamental desta revisão é responder à seguinte Questão de Pesquisa Principal (RQ):

RQ: De que forma a automação da análise estática de segurança (SAST) em pipelines de CI/CD contribui para reduzir custos, tempo de correção e riscos de exposição de vulnerabilidades do software em produção?

Para desdobrar e aprofundar a investigação, foram definidas três Questões de Pesquisa Secundárias:

RQ1: Como a identificação imediata de falhas de segurança nas fases iniciais do desenvolvimento (Shift Left) contribui para a agilidade da correção e a redução do acúmulo de erros?

RQ2: Quais critérios podem ser usados para categorizar ferramentas SAST de acordo com diferentes contextos, como linguagens suportadas, tipos de vulnerabilidades detectadas, facilidade de configuração, modelo de licenciamento e recursos de integração?

RQ3: Como a integração de ferramentas de SAST em pipelines de CI/CD impacta a qualidade da verificação de segurança no ciclo de desenvolvimento de software?

Essa revisão da literatura tem como objetivo apresentar técnicas e ferramentas de segurança que podem ser implementadas nos sistemas de automação de pipelines de software. Por meio de estudos de caso do mundo real e da literatura recente, exploramos as principais técnicas de segurança aplicadas ao desenvolvimento ágil. Essas técnicas incluem a análise estática de código (SAST), a verificação de vulnerabilidades em dependências externas (SCA) com foco na rastreabilidade por meio de listas de materiais de software (SBOM) e a análise dinâmica de vulnerabilidades (DAST) no ambiente de homologação. Além de delinear as ferramentas fundamentais para operacionalizar tais controles, o trabalho analisa de que forma a automação da segurança é capaz de reduzir o surgimento de vulnerabilidades críticas no ambiente de produção e otimizar sua correção de forma antecipada.

2. Revisão da literatura

A literatura recente reforça que DevSecOps e a abordagem Shift Left tornam a segurança mais eficaz quando integrada e automatizada em todas as etapas do pipeline CI/CD. E quando aplicada de forma correta o conceito de segurança, que era erroneamente posicionada como um estágio posterior no desenvolvimento de software, torna-se atualmente um pilar estratégico essencial no atual cenário tecnológico.

Artigo 1 - Implementation of DevSecOps by Integrating Static and Dynamic Security Testing in CI/CD Pipelines (2022) [Putra and Kabetta 2022]: Este artigo tem como objetivo integrar segurança proativa em ambientes ágeis para reduzir o tempo de lançamento e garantir que vulnerabilidades sejam detectadas precocemente. Estudo de caso utilizando ferramentas como GitLab CI/CD, Docker, onde foram automatizados testes de análise estática, com NJSSCAN e dinâmica com OWASP ZAP em um sistema web desenvolvido com Node.js e Flutter. Com automação o tempo de implantação foi reduzido de várias horas para minutos, identificando com sucesso vulnerabilidades críticas antes do

lançamento. A principal contribuição do artigo é demonstrar a eficácia prática da abordagem shift-Left, provando que a automação da segurança em pipelines CI/CD aumenta a eficiência do desenvolvedor sem comprometer a qualidade ou a proteção do código.

Artigo 2 - Automated security testing in DevSecOps pipelines: Integrating AI-based vulnerability discovery and compliance validation (2024) [Abdiukov 2024]: O artigo tem como objetivo investigar o papel essencial dos testes de segurança automatizados em ambientes de software ágeis e modernos, com foco em como a Inteligência Artificial pode ser utilizada para identificar vulnerabilidades e verificar a conformidade regulatória em pipelines DevSecOps. Estudo de revisão que avalia ferramentas de teste (SAST, DAST, e IAST) e processos de conformidade automatizados (GDPR, HIPAA, NIST) integrados à IA. O estudo conclui que a IA melhora a precisão na detecção de ameaças, reduz falsos positivos e permite uma postura de segurança proativa e escalável. A principal contribuição do artigo é apresentar o uso da IA como essencial para lidar com a complexidade regulatória moderna e a sofisticação das ciberameaças em ambientes ágeis.

Artigo 03- Semgrep: Improving the Limited Performance of Static Application Security Testing (SAST) Tools [Bennett et al. 2024]: O objetivo do artigo é avaliar e melhorar a detecção de ferramentas SAST populares (CodeQL, Snyk, Semgrep, FindSecBugs) em falhas reais. O artigo faz análise manual de falhas perdidas e criação de 29 novas regras customizadas para o Semgrep. O estudo conclui que o Semgrep* (versão aprimorada) aumentou a detecção em 181 por cento, superando a união de todas as outras ferramentas juntas. A contribuição principal do estudo demonstra que regras personalizadas são mais eficientes que empilhar múltiplas ferramentas padrão no pipeline.

Artigo 04 - Barriers to Using Static Application Security Testing (SAST) Tools: A Literature Review (2024) [Wadhams et al. 2024]: O estudo tem como objetivo identificar por que muitos desenvolvedores hesitam em adotar ferramentas de análise estática (SAST) em seus fluxos de trabalho e quais são os problemas específicos que levam ao abandono dessas ferramentas. O método de pesquisa utilizado no artigo se trata de uma revisão sistemática de literatura analisando 89 artigos publicados entre 2019 e 2023 focados na experiência do desenvolvedor com SAST. Os resultados indicam que a maior barreira foi a alta taxa de falso positivos, o que gera fadiga de alertas e perda de confiança na ferramenta, outras barreiras incluem saídas de dados confusas, configuração demorada e falta de sugestões automáticas de correção. A principal contribuição do estudo é mostrar que a adoção não depende não apenas de melhorias técnicas, mas de uma melhor compreensão dos fatores humanos e da usabilidade para os desenvolvedores.

Artigo 05- Comparison of Static Application Security Testing Tools and Large Language Models for Repo-level Vulnerability Detection [Zhou et al. 2024]: O objetivo do artigo é comparar a eficácia de 15 ferramentas tradicionais de SAST contra 12 modelos de linguagem de grande escala (LLMs) na detecção de vulnerabilidades em nível de repositório. O método de pesquisa desse artigo se trata de um estudo experimental comparativo utilizando conjunto de dados de vulnerabilidade reais em Java, C e Python, analisando taxas de detecção e falsos positivos. O estudo conclui que ferramentas SAST tradicionais possuem baixas taxas de detecção, mas poucos falsos positivos. Já os LLMs detectam entre 90 e 100 por cento das vulnerabilidades, porém sofrem com taxas de falsos positivos extremamente altas.

Artigo 06- DevSecOps: Integrating Security Into the DevOps Lifecycle with AI and Automation [Thopalle 2024]: O artigo analisa a integração de protocolos de segurança automatizados e IA para reduzir vulnerabilidades dentro de um pipeline CI/CD. O estudo analisa os fluxos de trabalho DevSecOps e o uso de Processamento de Linguagem Natural (NLP) e modelos como CodeBERT para análise semântica de código. A implementação reduziu o tempo de resposta a incidentes em 45 por cento e diminuiu em 60 por cento no número de vulnerabilidade que chegavam ao ambiente de produção. A principal contribuição é mostrar o impacto quantitativo da automação e da IA na criação de sistemas de software mais resilientes e autônomos.

Artigo 07- Static Analysis Techniques for Embedded, Cyber-Physical, and Electronic Software Systems [Iavich et al. 2026]: A Comprehensive Survey: O artigo tem como objetivo fornecer uma visão abrangente sobre técnicas de análises estáticas aplicadas em sistemas embarcados, sistemas ciberfísicos e softwares eletrônicos, destacando métodos, aplicações, desafios e tendências futuras. O estudo realiza uma revisão sistemática da literatura seguindo as diretrizes PRISMA, analisando publicações entre 2015 e 2025. O estudo desenvolveu uma taxonomia que mapeia técnicas (análise de fluxo de dados, modelagem, etc.) aos seus domínios de aplicações e analisa os compromissos técnicos entre integridade, precisão e escalabilidade. A principal contribuição do artigo é oferecer um guia estruturado para orientar pesquisas futuras e práticas industriais no desenvolvimento de sistemas eletrônicos críticos e confiáveis, destacando tendências como a análise híbrida estática-dinâmica

3. Metodologia

A presente revisão da literatura foi conduzida seguindo as diretrizes metodológicas propostas por Barbara Kitchenham. O protocolo do estudo foi estruturado e definido de forma colaborativa pelos integrantes do grupo. Durante a fase de planejamento, determinou-se o tema central da pesquisa, a Questão de Pesquisa Principal (RQ), as três Questões de Pesquisa Secundárias e as strings de busca. Adicionalmente, estabeleceu-se o processo de seleção dos artigos de acordo com critérios claros de elegibilidade, garantindo a estrita coerência e o alinhamento dos dados com o tema investigado.

3.1. Critérios de Elegibilidade

Para guiar a seleção dos estudos, estruturou-se o modelo PICOS da seguinte forma:

População: Ciclo de Vida de Desenvolvimento de Software (SDLC), especificamente ambientes que utilizam automação e pipelines de CI/CD.

Intervenção: Automação da análise estática de segurança (SAST), dinâmica (DAST) e adoção de práticas baseadas em Shift Left.

Comparação: Diferentes ferramentas SAST de mercado, abordagens tradicionais de segurança vs. automatizadas e o uso de SAST.

Resultados (Outcomes): Redução de custos e tempo de correção de bugs de segurança, mitigação de riscos em produção, melhoria na qualidade da verificação e agilidade no desenvolvimento.

Tipo de estudo: Revisões de literatura, estudos empíricos, pesquisas de opinião da indústria, benchmarks e relatórios de conferências científicas.

3.2. Estratégia de Busca

Foram elaboradas 4 strings de busca específicas, alinhadas diretamente às RQs do projeto:

- 1 - ("SAST"OR "Static Application Security Testing") AND ("CI/CD"OR "continuous integration"OR "continuous delivery") AND ("vulnerability"OR "security")
- 2 - ("Shift Left"OR "shift-left security") AND ("DevSecOps"OR "secure development") AND ("vulnerability detection"OR "early detection"OR "SDLC")
- 3 - ("SAST tools"OR "static analysis tools") AND (comparison OR evaluation OR benchmark) AND ("vulnerability detection"OR "false positive"OR "security testing")
- 4 - (DevSecOps ("DevSecOps") AND ("automation"OR "automated security testing") AND ("pipeline"OR "software development lifecycle"OR "SDLC"))

3.3. Extração de Dados

Os dados foram coletados e organizados de forma sistemática a partir das informações principais dos artigos selecionados. Com o objetivo de fornecer subsídios para responder diretamente às questões de pesquisa (RQs) estabelecidas, foram extraídos os seguintes elementos de cada estudo: Título do artigo, Nome dos autores, Ano de publicação, Fonte de publicação (conferência, periódico ou indexador científico), Relação com DevOps/DevSecOps (integração de segurança e abordagem Shift Left em pipelines de CI/CD), Contribuição do artigo para a revisão (principais achados sobre redução de custos, tempo de correção e mitigação de vulnerabilidades).

3.3.1. Tabela de Evidências

Artigo	Ano	Prática DevOps	Ferramentas	Contexto	Resultados
Implementation of DevSecOps by Integrating Static and Dynamic Security Testing in CI/CD Pipelines	2024	CI/CD, Shift Left, Automated Security Testing	GitLab CI/CD, Docker, NJSSCAN, OWASP ZAP	Aplicação Web (Node.js e Flutter)	Redução do tempo de implantação de horas para minutos e identificação de vulnerabilidades críticas antes do lançamento
Automated Security Testing in DevSecOps Pipelines: Integrating AI-Based Vulnerability Discovery and Compliance Validation	2025	Automated Security Testing, Compliance Automation, DevSecOps	IA, SAST, DAST, IAST	Ambientes sujeitos a conformidade (GDPR, HIPAA, NIST)	Melhoria da precisão na detecção de ameaças, redução de falsos positivos e postura de segurança mais proativa
Semgrep*: Improving the Limited Performance of Static Application Security Testing (SAST) Tools	2024	Automated Security Testing, Continuous Security Validation	Semgrep*, CodeQL, Snyk, FindSecBugs	Benchmark de ferramentas SAST	Aumento de 181% na capacidade de detecção através da customização de regras
Barriers to Using Static Application Security Testing (SAST) Tools: A Literature Review	2024	Shift Left Security, Automated Security Testing	Ferramentas SAST (geral)	Revisão sistemática sobre adoção de SAST	Identificação dos falsos positivos como principal barreira à adoção, além de problemas de usabilidade e configuração
Comparison of Static Application Security Testing Tools and Large Language Models for Repo-Level Vulnerability Detection	2025	Automated Security Testing, Vulnerability Detection	Ferramentas SAST e LLMs	Repositórios reais em Java, C e Python	LLMs apresentaram maior taxa de detecção, porém com muitos falsos positivos; SAST apresentou menor detecção e maior precisão

Artigo	Ano	Prática DevOps	Ferramentas	Contexto	Resultados
DevSecOps: Integrating Security Into the DevOps Lifecycle with AI and Automation	2025	DevSecOps, Shift Left, Security Automation	IA, NLP, CodeBERT	Pipeline CI/CD automatizado	Redução de 45% no tempo de resposta a incidentes e 60% menos vulnerabilidades chegando à produção
Static Analysis Techniques for Embedded, Cyber-Physical, and Electronic Software Systems: A Comprehensive Survey	2025	Static Analysis, Continuous Security Validation	Técnicas de análise estática e híbrida	Sistemas embarcados, ciberfísicos e eletrônicos	Desenvolvimento de taxonomia das técnicas e identificação de tendências futuras para análise estática e híbrida

Tabela 1. Tabela de evidências

3.3.2. Práticas Identificadas

Prática DevOps / DevSecOps	Descrição da Prática no Contexto dos Estudos	Artigos Relacionados
Automação de Pipelines CI/CD	Integração e Entrega Contínuas automatizadas para acelerar o ciclo de vida do software, reduzindo o tempo de deploy de horas para minutos.	Artigo 1, Artigo 2, Artigo 3, Artigo 6
Abordagem Shift Left	Antecipação dos testes e validações de segurança para as fases iniciais do desenvolvimento, mitigando riscos antes que o código chegue à produção.	Artigo 1, Artigo 2, Artigo 4, Artigo 6
Testes Estáticos de Segurança (SAST)	Análise automatizada do código-fonte ou binários sem a necessidade de executar a aplicação, identificando falhas estruturais e lógicas.	Artigo 1, Artigo 2, Artigo 3, Artigo 4, Artigo 5, Artigo 7
Testes Dinâmicos de Segurança (DAST / IAST)	Avaliação da segurança da aplicação em tempo de execução (homologação/pré-produção), simulando ataques reais para validar configurações e sessões.	Artigo 1, Artigo 2, Artigo 7
Customização de Regras de Segurança	Prática de adaptar e criar regras específicas para os motores de análise (ex.: Semgrep), priorizando o contexto do projeto em vez do empilhamento de ferramentas padrão.	Artigo 3
Segurança Impulsionada por IA e LLMs	Uso de Inteligência Artificial, Modelos de Linguagem de Grande Escala e NLP (ex.: CodeBERT) para conduzir análises semânticas avançadas e detecção em nível de repositório.	Artigo 2, Artigo 5, Artigo 6
Conformidade Automatizada (Compliance as Code)	Validação automática e contínua de requisitos regulatórios e de governança (como GDPR, HIPAA e NIST) diretamente no fluxo de entrega do software.	Artigo 2

Prática DevOps / DevSecOps	Descrição da Prática no Contexto dos Estudos	Artigos Relacionados
Mitigação de Atrito e Fadiga de Alertas (Foco em Developer Experience)	Gestão de falsos positivos, melhoria na clareza dos relatórios técnicos e adoção de sugestões automáticas de correção (auto-remediation) para engajar os desenvolvedores.	Artigo 4
Análise Híbrida de Segurança	Combinação sinérgica de métodos estáticos e dinâmicos para equilibrar a integridade, a precisão e a escalabilidade, especialmente em sistemas críticos.	Artigo 7
Containerização e Padronização	Uso de ferramentas de isolamento de ambiente (Docker) para garantir consistência e reprodutibilidade nos testes e implantações em pipelines.	Artigo 1

Tabela 2. Práticas DevOps e DevSecOps identificadas nos estudos selecionados

3.3.3. Ferramentas Identificadas

Ferramenta / Tecnologia	Categoria / Função	Artigo(s) Relacionado(s)	Contexto de Uso e Desempenho no Estudo
GitLab CI/CD	Orquestração e Automação de Pipelines	Artigo 1	Utilizado para automatizar o fluxo de integração e entrega contínua, reduzindo o tempo de deploy para minutos.
Docker	Containerização e Infraestrutura	Artigo 1	Responsável pelo isolamento e consistência dos ambientes onde os testes automatizados foram executados.
Semgrep (e Semgrep*)	Análise Estática de Segurança (SAST)	Artigo 03	Teve seu motor otimizado com 29 regras customizadas, gerando um aumento de 181% na detecção de falhas reais.
CodeQL	Análise Estática de Segurança (SAST)	Artigo 03	Ferramenta tradicional baseada em queries semanticamente ricas, utilizada no benchmark comparativo.
Snyk	Análise Estática (SAST) e Composição de Software (SCA)	Artigo 03	Ferramenta de mercado utilizada no benchmark para avaliar a eficácia contra falhas de segurança reais.

Ferramenta / Tecnologia	Categoria / Função	Artigo(s) Relacionado(s)	Contexto de Uso e Desempenho no Estudo
FindSecBugs	Análise Estática de Segurança (SAST)	Artigo 03	Plugin de auditoria focado especificamente em aplicações Java/JVM, utilizado no benchmark do estudo.
NJSSCAN	Análise Estática de Segurança (SAST)	Artigo 1	Ferramenta SAST especializada em encontrar vulnerabilidades de código em aplicações Node.js.
OWASP ZAP	Teste Dinâmico de Segurança (DAST)	Artigo 1	Utilizado para realizar varreduras ativas e passivas na aplicação web em tempo de execução (homologação).
CodeBERT	Modelo de IA / Processamento de Linguagem Natural	Artigo 06	Modelo de aprendizado profundo utilizado para realizar análise semântica avançada de código-fonte.
Modelos de Linguagem de Grande Escala (LLMs)	Inteligência Artificial Generativa	Artigo 05	Grupo de 12 modelos (como GPT similares) testados para detecção em nível de repositório. Alcançaram 90–100% de detecção, mas com alto índice de falsos positivos.
Node.js / Flutter	Tecnologias de Desenvolvimento (Stack)	Artigo 1	Tecnologias que compunham o sistema web do estudo de caso real (Backend em Node.js e Frontend/Mobile em Flutter).

Tabela 3. Ferramentas e tecnologias identificadas nos estudos analisados

4. Resultados e Discussões

A partir da revisão sistemática da literatura conduzida conforme o protocolo de Kitchenham e dos critérios de elegibilidade estabelecidos, foram analisados artigos que abordam a automação de segurança em pipelines CI/CD, com foco nas técnicas SAST, SCA, DAST e na abordagem Shift Left.

4.1. Eficácia da automação na detecção precoce de vulnerabilidades

Os resultados indicam que a integração de ferramentas automatizadas de análise estática (SAST) e dinâmica (DAST) em pipelines CI/CD reduz significativamente o tempo de detecção de vulnerabilidades. No estudo de caso do Artigo 1, a automação reduziu o tempo de implantação de várias horas para minutos, identificando vulnerabilidades críticas antes da liberação para produção. Esse achado corrobora a premissa da abordagem Shift Left, na qual a segurança é antecipada às fases iniciais do ciclo de vida.

4.2. Barreiras de Adoção e o Fator Humano

Apesar do expressivo avanço tecnológico, os resultados apontam que o elo mais fraco da cadeia DevSecOps reside na experiência do desenvolvedor. A revisão sistemática de 89 artigos conduzida pelo Artigo 04 (período de 2019 a 2023) identificou que a alta taxa de falsos positivos é a principal responsável pelo abandono ou subutilização de ferramentas SAST.

Esse fenômeno, conhecido como fadiga de alertas, corrói a confiança do time de engenharia de software na automação. Quando combinada com saídas de dados confusas, configurações de ambiente demoradas e a ausência de recursos de sugestão automática de correção (auto-remediation), a segurança passa a ser vista pelas equipes ágeis como um elemento de atrito operacional, e não como um pilar estratégico.

4.3. Comparação entre SAST, DAST e SCA

Os artigos revisados apontam que nenhuma técnica isolada é suficiente para garantir a segurança em pipelines CI/CD. A combinação sinérgica entre SAST (análise estática de código-fonte), SCA (verificação de dependências externas com rastreabilidade via SBOM) e DAST (testes dinâmicos em ambiente de homologação) proporciona uma cobertura mais ampla de vulnerabilidades. Contudo, a literatura ainda carece de benchmarks padronizados que permitam comparar diretamente a eficácia relativa dessas ferramentas em diferentes contextos organizacionais.

4.4. Redução de custos e tempo de correção

Os achados sugerem que a automação de segurança reduz o custo e o tempo de correção de vulnerabilidades quando comparada a modelos tradicionais (segurança como etapa final isolada). A detecção precoce, ainda nas fases de codificação e compilação, evita que falhas críticas cheguem ao ambiente de produção, mitigando riscos de incidentes e retrabalho tardio. No entanto, os artigos analisados não apresentam valores monetários ou métricas de tempo padronizadas, limitando generalizações mais precisas.

4.5. Discussão geral

Os resultados confirmam que a automação da segurança em pipelines CI/CD, alinhada aos princípios DevSecOps, é eficaz para reduzir vulnerabilidades em produção e acelerar a correção. Contudo, a literatura revela uma lacuna significativa: o foco excessivo em aspectos técnicos em detrimento dos fatores humanos. A alta taxa de falsos positivos, por exemplo, não é apenas um problema de precisão algorítmica, mas uma barreira comportamental que compromete a adoção contínua pelas equipes de desenvolvimento.

Além disso, embora a IA seja apontada como promissora, os estudos ainda carecem de validação empírica em larga escala e de métricas objetivas de retorno sobre o investimento. Recomenda-se que futuras pesquisas investiguem modelos híbridos de SAST com aprendizado de máquina para redução de falsos positivos, bem como a integração de feedback loops contínuos com os desenvolvedores.

5. Conclusão

Esta revisão literatura investigou a automação e a integração de técnicas de segurança em pipelines de desenvolvimento de software, fundamentando as vantagens e os desafios práticos da consolidação da cultura DevSecOps sob a égide da abordagem Shift Left.

As conclusões deste estudo indicam que a automação síncrona de técnicas como SAST e DAST é um mecanismo altamente eficaz, capaz de mitigar as vulnerabilidades antes que o código atinja o ambiente de produção, além de acelerar o tempo de resposta a incidentes. No entanto, o estudo revela uma dualidade técnica importante: enquanto abordagens modernas baseadas em LLMs oferecem taxas de cobertura quase completas, elas introduzem um volume insustentável de falsos positivos. Essa característica agrava o cenário de fadiga de alertas e as barreiras de usabilidade que historicamente levam os desenvolvedores a rejeitar ou abandonar os mecanismos de segurança estática.

Como principal contribuição acadêmica e prática, este artigo demonstra que a evolução do DevSecOps não deve se concentrar no mero empilhamento de ferramentas padronizadas de mercado. Em vez disso, o caminho para uma postura de segurança escalável e resiliente passa pela customização de regras contextuais e pelo emprego de modelos híbridos de análise (estática-dinâmica) refinados por algoritmos de IA semântica que reduzam o ruído técnico apresentado às equipes humanas.

Referências

- Abdiukov, T. (2024). Automated security testing in devsecops pipelines: Integrating ai-based vulnerability discovery and compliance validation. *World Journal of Advanced Research and Reviews*, pages 2083–2093.
- Bennett, G., Hall, T., Winter, E., and Counsell, S. (2024). Semgrep*: Improving the limited performance of static application security testing (sast) tools. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, EASE '24, page 614–623, New York, NY, USA. Association for Computing Machinery.
- Iavich, M., Kuchukhidze, T., and Lopata, A. (2026). Static analysis techniques for embedded, cyber-physical, and electronic software systems: A comprehensive survey. *Electronics*, 15(5).
- Putra, A. M. and Kabetta, H. (2022). Implementation of devsecops by integrating static and dynamic security testing in ci/cd pipelines. In *2022 IEEE International Conference of Computer Science and Information Technology (ICOSNIKOM)*, pages 1–6.
- Thopalle, P. K. (2024). Devsecops: Integrating security into the devops lifecycle with ai and automation. *International Journal of Advanced Research in Engineering and Technology (IJARET)*, pages 452–466.
- Wadhams, Z. D., Izurieta, C., and Reinhold, A. M. (2024). Barriers to using static application security testing (sast) tools: A literature review. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops*, ASEW '24, page 161–166, New York, NY, USA. Association for Computing Machinery.
- Zhou, X., Tran, D.-M., Le-Cong, T., Zhang, T., Irsan, I. C., Sumarlin, J., Le, B., and Lo, D. (2024). Comparison of static application security testing tools and large language models for repo-level vulnerability detection.